

Web Development for Whitman County Veteran Services

**Sponsored By:
Becky Buri, Veterans Service Officer**

**Faculty Mentor:
Dr. Parteek Kumar, PhD**

**By:
Logan Cribbs, Chhi Ly**



School of Electrical Engineering & Computer Science

Voiland College of Engineering and Architecture

Washington State University

Pullman, Washington

October 2025

TABLE OF CONTENTS

I. Introduction	2
II. System Overview	2
III. Architecture Design	3
III.1. Overview	3
III.2. Subsystem Decomposition	3
III.2.1. Next.js Frontend	4
III.2.2. API Routes / Server Logic	5
III.2.3. Authentication Subsystem	6
III.2.4. Prisma Database Layer	7
III.2.5. File Management & Flyer Handler	8
IV. Data design	9
V. User Interface Design	11
VI. Constraints	12
VII. Glossary	13
VIII. References	13
VIII. Appendices	14

I. Introduction

The purpose of this document is to present the proposed solution approach for the Whitman County Veteran Services (WCVS) Veteran e-Post Hub, a community-driven web application designed to provide veterans and their families with access to local events, resources, and announcements. This document describes the architectural design, core subsystems, and technologies that will be used to implement the system. It serves as a reference for developers, project stakeholders, and future maintainers who will continue development after the initial prototype phase.

The overall goal of the WCVS Veteran e-Post Hub is to create a centralized, easy-to-use online platform that connects local veterans to information and events in their area while enabling administrators to efficiently post, manage, and update event content.

II. System Overview

The WCVS e-Post Hub follows a three-tier web architecture consisting of a frontend interface, backend server logic, and database layer. The system is implemented using a Next.js + Tailwind CSS + Prisma + SQLite stack, which supports efficient rendering, reliable data storage, and intuitive design.

The system allows two main user types:

- **Veterans / General Users:** Can browse and download event flyers, access resource links, and view event details.
- **Administrators:** Can log in to a secure dashboard to create, edit, and delete event posts and flyers.

All event data is stored in an SQLite database using Prisma ORM for structured access and validation. Vercel is used for deployment and continuous integration, providing automatic builds and easy synchronization with the project's GitHub repository.

Key features of the system include:

- Event posting with downloadable PDF flyers.
- A secure login for administrative functions.
- Responsive layout across devices (mobile, tablet, desktop).
- Consistent and accessible design following WCVS branding.
- Low-cost hosting with built-in scalability and maintenance simplicity.

The architecture was designed with long-term maintainability and clarity in mind, ensuring that new developers can easily contribute or modify features as the WCVS site evolves.

III. Architecture Design

III.1. Overview

The WCVS e-Post Hub uses a modular and layered architecture based on the client–server model, which divides the system into independent layers that communicate through clearly defined interfaces. This structure improves maintainability and testing efficiency, allowing frontend and backend components to evolve independently.

The architectural pattern adopted is a Model–View–Controller (MVC) variation implemented through Next.js, where:

- The view is handled by React components and Tailwind CSS layouts handle the View.
- The controller logic is implemented in Next.js API routes that process user actions and inputs.
- The model is defined through Prisma's schema, managing data persistence and integrity.

This pattern fits the project well because it supports high cohesion within each layer and low coupling between the frontend and backend. It allows developers to work on UI improvements, event management features, or database enhancements independently, minimizing merge conflicts and simplifying debugging.

III.2. Subsystem Decomposition

The system is decomposed into five major subsystems:

1. **Next.js Frontend**
2. **API Routes / Server Logic**
3. **Authentication Subsystem**
4. **Prisma Database Layer**
5. **File Management and Flyer Handler**

Below is a UML component diagram that illustrates the system's subsystem decomposition for the WCVS e-Post webpage. The diagram shows how the major components interact within the system architecture. Each subsystem is described in detail below, including its responsibilities, concepts, interfaces, and role in the overall architecture.

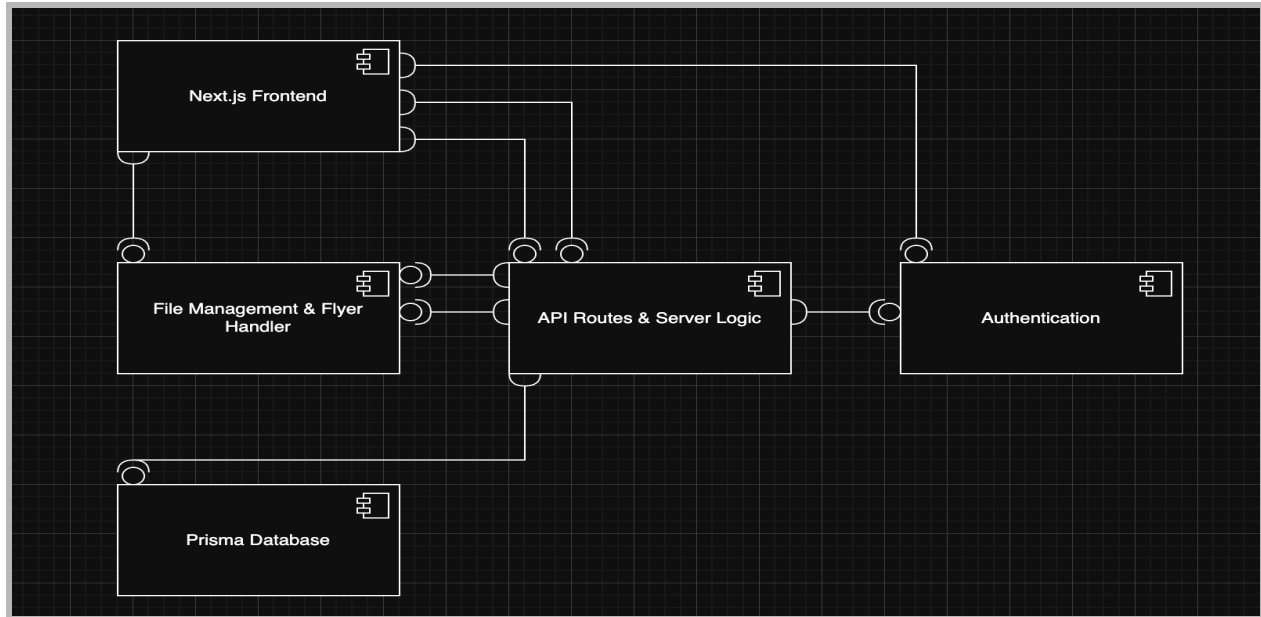


Figure 1 - Subsystem Decomposition

III.2.1 Next.js Frontend

a) *Description*

The Next.js Frontend represents the user-facing interface of the e-Post Hub. It is built using React components, Tailwind CSS, and HeroUI. The frontend is responsible for rendering dynamic pages, handling user interactions, and providing a consistent design across all screen sizes.

It includes the following key pages:

- **Home Page:** General introduction and quick access to current events.
- **Events Page:** Displays event listings with thumbnails, descriptions, and flyer download buttons.
- **Admin Login & Dashboard:** Allows authorized users to log in and manage events.

b) *Concepts and Algorithms Generated*

The frontend uses a component-based architecture for modularity. Components such as EventCard, Navbar, and EventForm are reusable across multiple pages. The data flow follows React's state management principles, using local state for form handling and the SWR (stale-while-revalidate) library for real-time event data updates.

Dynamic routing is implemented through Next.js's file-based routing, which automatically maps URLs to page components. The decision to use Next.js instead of plain React was made because it supports server-side rendering (SSR) for improved SEO and faster load times, particularly for the public-facing event pages.

c) Interface Description

Services Provided:

Service Name	Provided To	Description
Render Pages	Users / Admins	Displays interactive pages (Home, Events, Admin) using responsive layouts.
Send Form Data	API Routes	Sends user input (login credentials, event forms) to backend endpoints.
Display Event Data	Users	Fetches event data and renders it as formatted cards or tables.

Services Required:

- Event data from API Routes
- Authentication tokens from Auth Subsystem
- Flyer URLs from File Handler

III.2.2 API Routes / Server Logic

a) Description

The API Routes act as the bridge between the frontend and the backend. Implemented through Next.js's built-in `/api` directory, these routes handle all communication, data validation, and logic for *CRUD* operations.

Key endpoints include:

- `/api/events` → Get all approved events.
- `/api/event/create` → Add new event (admin only).
- `/api/event/edit` and `/api/event/delete` → Update or remove existing events.
- `/api/auth/login` → Verify user credentials.

b) Concepts and Algorithms Generated

Each route follows a RESTful design pattern, maintaining consistency across the API. The routes use *Zod* for input validation to ensure proper data structure before updating the database. Error handling returns clear HTTP codes and JSON responses for frontend interpretation.

Middleware is used to validate authentication tokens and restrict admin operations. This provides a security layer without requiring an external server or framework.

c) Interface Description

Services Provided

Service Name	Provided To	Description
Fetch Events	Frontend	Retrieves all event data and returns JSON for rendering.
Create Event	Admin Dashboard	Adds new event details to the database after validation.
Update Event	Admin Dashboard	Edits event information and updates associated flyers.
Delete Event	Admin Dashboard	Removes events and cleans up linked flyer references.

Services Required

- **Database access** (via Prisma)
- **User verification** (from Auth Subsystem)
- **Flyer upload / delete** (from File Handler)

III.2.3 Authentication Subsystem

a) Description

The Authentication subsystem manages secure logins and session control for administrative users. It ensures that only authorized personnel can modify event data or upload new flyers.

b) Concepts and Algorithms Generated

The subsystem uses *bcrypt* to hash and verify passwords. Authentication requests are processed through `/api/auth/login`, where user credentials are compared to hashed records in the database. Upon successful login, NextAuth.js (or JWT tokens) maintain the session securely using HTTP-only cookies.

This design prioritizes simplicity and portability while still adhering to secure authentication practices.

c) Interface Description

Services Provided

Service Name	Provided To	Description
Validate Login	API / Frontend	Authenticates user credentials and issues session token.
Authorize Access	API Routes	Confirms token validity for admin-only operations.
Logout Session	Frontend	Ends current session and clears cookie/token.

Services Required

- **User record retrieval** (from Prisma Database Layer)

III. 2. 4 Prisma Database Layer

a) Description

The Prisma Database Layer acts as the system's data management core. It provides a type-safe interface for interacting with the underlying SQLite database. Prisma automatically generates functions to perform *CRUD* operations based on the schema.

b) Concepts and Algorithms Generated

The schema defines three main models:

- **User:** Contains username, hashed password, and role (admin).
- **Event:** Stores title, description, date, and flyer URL.
- **ResourceLink:** Stores external resource references for veterans.

Migrations are handled automatically through Prisma CLI commands. During deployment, the same schema can be easily adapted for PostgreSQL on Vercel to support concurrent connections.

c) Interface Description

Services Provided

Service Name	Provided To	Description
Retrieve Events	API Routes	Returns all event entries for frontend display.
Insert Event	API Routes	Adds new event records to the database.
Update Event	API Routes	Modifies event entries (title, date, or flyer).
Delete Event	API Routes	Removes event data when deleted by admin.
Authenticate User	Auth Subsystem	Retrieves user information and hashed credentials.

Services Required

- None (accessed internally through Prisma Client).

III. 2. 5 File Management & File Handler

a) Description

This subsystem manages all flyer and document uploads. It ensures each event's flyer maintains uniform formatting and downloadable functionality. Files are temporarily processed and linked to their corresponding event database entry.

b) Concepts and Algorithms Generated

File uploads are validated for type (.pdf, .jpg, .png) and size before being stored. Each upload triggers a function to update the associated database record with a public URL reference. Flyer downloads use standard HTTP GET requests with built-in browser PDF handling.

For future scalability, the design can integrate cloud storage such as AWS S3 or Vercel Blob to replace local handling if needed.

c) Interface Description

Services Provided

Service Name	Provided To	Description
Upload Flyer	Admin Dashboard / API	Accepts uploaded event flyer and returns public link.
Retrieve Flyer	Frontend	Provides download or preview capability for users.
Delete Flyer	Admin Dashboard	Removes flyer file and clears record on deletion.

Services Required

- **Database Update** (via Prisma)
- **Auth Verification** (via Authentication Subsystem)

IV. Data design

The data design for the Whitman County Veteran Services (WCVS) e-Post Hub focuses on creating a structured, efficient, and maintainable schema to support both administrative and user-facing functionality. The system employs Prisma ORM to define the schema and interact with a lightweight SQLite database. This design allows for clear entity relationships, type safety, and consistent data validation throughout development.

1. Database Overview

The application follows a relational data model, where each table corresponds to a logical entity within the system. The schema supports *CRUD* operations (Create, Read, Update, Delete) across event and user data. By using Prisma's schema definition language, developers can easily synchronize the database with code-level models and maintain referential integrity.

2. Entities and Relationships

The major Entities in the system are:

- **User** – Stores information about administrators, including credentials and access privileges.
- **Event** – Represents community events, including titles, descriptions, and associated flyers.
- **Flyer** – Contains metadata about uploaded PDF flyers linked to specific events.
- **Resource** – Stores external links and references to veteran service organizations.
- **SessionToken** - Tracks active login sessions to maintain secure authentication with appropriate authorized actions.

Entity Relationships

- One **User** can create multiple **Events**.
- Each **Event** has one corresponding **Flyer** file.
- All **Resources** are independent and publicly accessible.
- One **User** can have multiple **SessionTokens**.

These relationships promote data normalization and minimize redundancy, ensuring each entity has a single, well-defined purpose within the database.

3. Data Validation and Integrity

All data undergoes validation both at the application and schema levels:

- *Zod* validation enforces correct data types and formats on API input.
- *Prisma* constraint ensures referential integrity between various entities.
- Timestamps ('createdAt', 'updatedAt') are automatically managed and maintained by *Prisma*.
- Input sanitization prevents injection or malformed requests.

4. Sample Prisma Schematic

```
model User {  
  id          Int           @id @default(autoincrement())  
  username    String        @unique  
  passwordHash String  
  role        Role          @default(ADMIN)  
  events      Event[]  
  sessions    SessionToken[]  
  createdAt   DateTime       @default(now())  
  updatedAt   DateTime       @updatedAt  
}
```

Figure IV.a. Sample schema for User (Admin)

```

model Flyer {
  id      Int      @id @default(autoincrement())
  filename String
  path    String
  uploadedAt DateTime @default(now())
  event   Event    @relation(fields: [eventId], references: [id])
  eventId Int      @unique
}

```

Figure IV.b. Sample schema for Flyer

```

model Resource {
  id      Int      @id @default(autoincrement())
  title   String
  link    String
  category String? //nullable
  createdAt DateTime @default(now())
}

```

Figure IV.c. Sample schema for Resource

V. User Interface Design

The user interface (UI) for the WCVS Veteran e-Post hub is designed to prioritize simplicity, accessibility, and consistency for both veterans and administrators. Built using Next.js, React components, and Tailwind CSS, the interface emphasizes a seamless and responsive experience across devices.

1. Design Principles

- **Consistency:** Uses Tailwind CSS and HeroUI for uniform design.
- **Accessibility:** Follows WCVS color palette and contrast guidelines.
- **Responsiveness:** Adapts across mobile, tablet, and desktop views.
- **Simplicity:** Reduces clutter and uses clear navigation for veterans unfamiliar with tech.

2. Key Pages and Components

- Home Page
 - Rebranding the look of the Veteran e-Post hub landing page.
 - Shows featured or upcoming events. Layout is more 'Intuitive'.
 - Contains directory/navigation links: Home, Resources, Contact, About.
- Events Page
 - Lists events as interactable cards with viewable thumbnail images.
 - Each card links to full event details that will no longer create a new tab.
- Admin Dashboard
 - Login form (username + password) strictly for Admin only.
 - After login: Contains *CRUD* panel for event management.
 Features include:
 - Add new event tile with description, and flyer upload.

- Edit or delete existing events.
- Live preview of event list.

3. Navigation Flow

- Users navigate from the home page to view events or access resources.
- Administrators log in to the dashboard to perform *CRUD* operations on event data.
- The navigation bar and footer remain persistent throughout all pages for consistency.

4. Visual Prototypes

Figma wireframes and mockups (see *Appendices* section) illustrate the envisioned final layout, color scheme, and responsive behavior of the WCVS e-Post Hub interface. These prototypes provide a tangible representation of the design direction and serve as a guiding reference throughout the development process.

The visual design emphasizes clarity, accessibility, and adherence to WCVS branding standards. Client feedback highlighted that the current layout provides a strong foundation for future enhancements, offering both aesthetic appeal and functional extensibility as the platform evolves.

VI. Constraints

1. Technical Constraints

- **Framework Limitations:** The system architecture is bound to the Next.js 14 framework and React 18 runtime, limiting backend flexibility compared to a full Node.js Express server.
- **Database Choice:** The initial deployment uses SQLite for simplicity and cost-efficiency; however, concurrent multi-admin editing is limited due to SQLite's file-based locking mechanism.
- **Hosting Environment:** Deployment on Vercel imposes execution-time limits on serverless functions (up to 10 seconds for free tier) and storage restrictions that may affect future scalability.

2. Environmental Constraints

- **Browser Compatibility:** The UI must perform consistently across modern browsers (Chrome, Edge, Firefox, Safari). Legacy browsers (IE 11 and below) are not supported.
- **Responsive Design:** All pages must auto-scale and remain fully functional across screen sizes from 320 px (mobile) to 1920 px (desktop).
- **Deployment Dependence:** System availability is tied to Vercel's uptime and GitHub synchronization for continuous deployment.

3. Operational Constraints

- **Administrative Access:** Only Whitman County Veteran Services authorized admins can log in to create, edit, or delete events.

- **Data Backup:** SQLite database backups must be scheduled manually before each major release.

4. Security and Privacy Constraints

- **Password Storage:** All passwords must be hashed with *bcrypt* before database insertion; plaintext passwords are strictly prohibited.
- **Transport Security:** All requests must be served over HTTPS via Vercel's built-in SSL encryption.
- **Data Retention:** Personally identifiable information is not collected from users beyond admin credentials.

5. Project Management Constraints

- **Timeline:** The prototype must be completed by December 5th, 2025, as a deliverable for the WSU capstone course.
- **Team Size:** Development is limited to two contributors, restricting parallel feature development and QA capacity.
- **Budget:** The project must operate within zero-cost or student discount tiers for all tools (Next.js, Vercel, GitHub, Prisma Cloud).

VII. Glossary

CRUD - Abbreviation for Create, Read, Update, Delete. In the context of this project *CRUD* operations are handled by API routes: POST, GET, PUT, DELETE respectively.

Zod - Typescript-first validation library used to ensure data requests and responses (e.g., from user forms or API requests) has the correct structure and type before it is processed or stored in the database.

bcrypt - password-hashing algorithm designed to securely store user passwords. It transforms strings into irreversible, random hashes that are safe to store in the database.

VIII. References

[1] A. Tailor, V. Shrivastava, A. Pandey, and S. Kumar, "Unraveling Next.js: A comprehensive overview," *Int. J. Res. Publ. Rev.*, vol. 5, no. 3, pp. 2675-2679, Mar. 2024. [Online]. Available: <https://ijrpr.com/uploads/V5ISSUE3/IJRPR23656.pdf>

[2] J. Ajayi, "Prisma ORM adoption guide: Overview, examples, and alternatives," *LogRocket Blog*, Sep. 2024. [Online]. Available: <https://blog.logrocket.com/prisma-orm-adoption-guide/>

VIII. Appendices

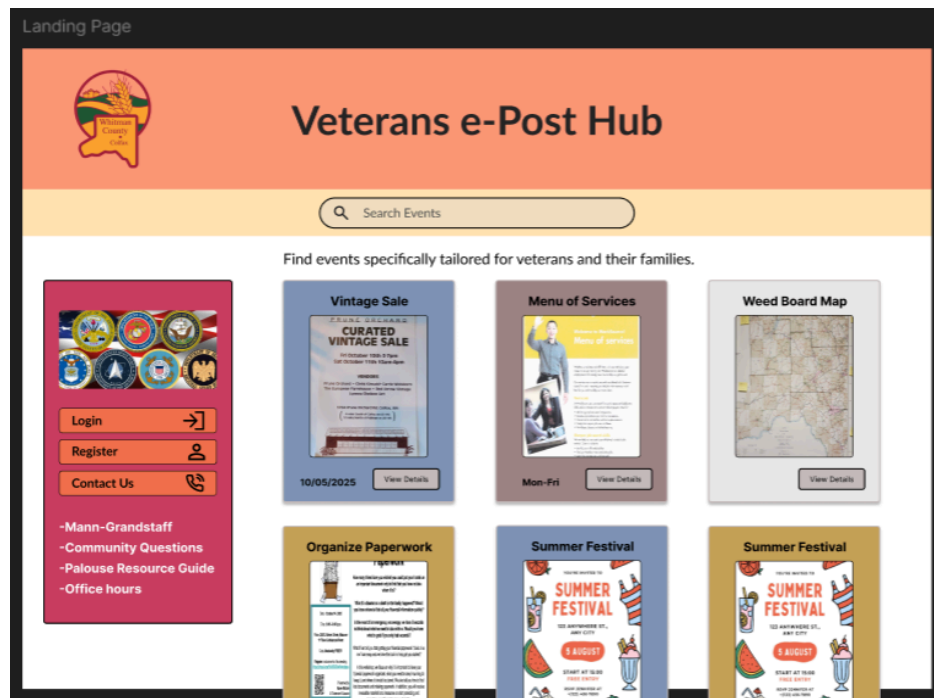


Figure VIII.a. Landing Page Mockup for Desktop UI

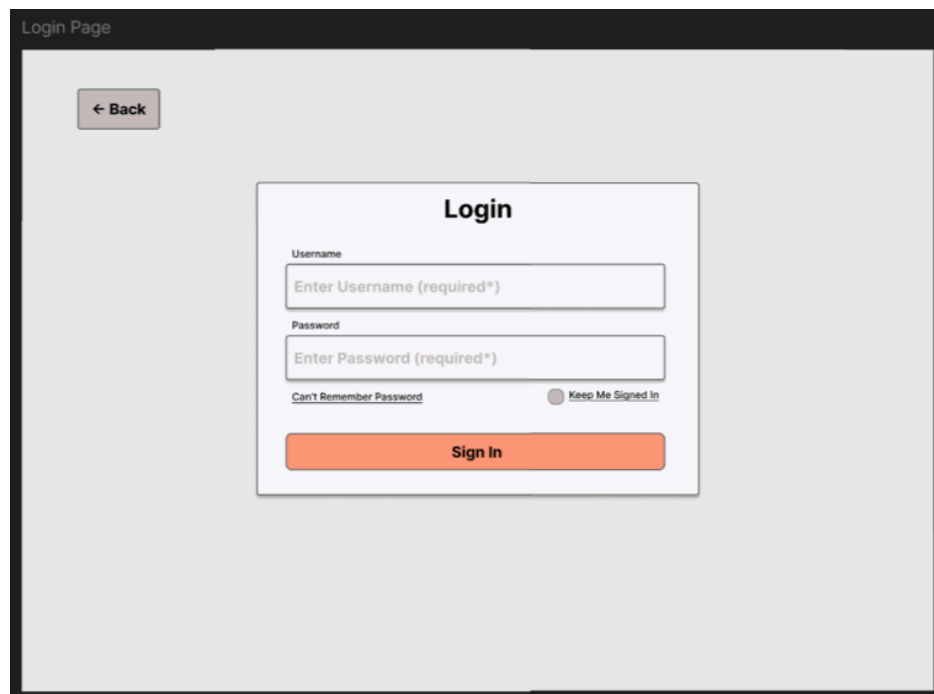


Figure VIII.b. Login Page for Admin on Desktop UI

Create New Event Page

← Back

Create New Event

Enter Event Title (Required*)

+ Add Dates

Enter Event Description (Optional)

Flyer / Attachment (Required*)

Choose File No File Selected

Event Address (Optional)

Event Address (Optional)

Submit Event

Figure VIII.c. Create Event page for Admin on Desktop UI

Home - Landing Page

Veterans e-Post Hub

Resources Community Questions Login

Welcome to the Whitman County Veterans Page

Find events specifically tailored for veterans and their families.

Search Events

17th Annual Autumn Festival
Saturday, October 4th & Sunday, October 5th

Event One [more info](#)

HALLOWEEN CARNIVAL
FREE ADMISSION!
October 21st

Event two [more info](#)

Login

Veterans e-Post Hub

Resources Community Questions Login

← Back

Welcome, user.

Username

Enter username

Password

Enter password

[Can't remember password](#) [Create an account](#)

login

Figure VIII.d. Landing page and Login page for Admin, respectively for Mobile UI.